RateKing Quick Start Guide

Pedro H. F. Matias

Version 0.1.0

1 Installation

Install RateKing using pip:

```
pip install RateKing
```

Verify installation:

```
import RateKing
print(RateKing.__version__)  # Should print: 0.1.0
```

2 Basic Example: $NH_3 + OH \rightarrow NH_2 + H_2O$

2.1 Step 1: Import Required Modules

```
from RateKing import Reaction, TST
from RateKing.tunneling_corrections import eckartCorrection
```

2.2 Step 2: Define Reaction Components

```
# Reactants
reactant1 = {
    "name": "NH3",
    "filePath": "path/to/nh3.out",
    "link": 2  # Or -1 for last link
}

reactant2 = {
    "name": "OH",
    "filePath": "path/to/oh.out",
    "link": 2
}

# Transition State
ts = {
    "name": "TS",
    "filePath": "path/to/ts.out",
    "link": 2
}

# Products
product1 = {
    "name": "NH2",
    "filePath": "path/to/nh2.out",
```

```
    "link": 2
}

product2 = {
    "name": "H2O",
    "filePath": "path/to/h2o.out",
    "link": 2
}
```

2.3 Step 3: Create and Run Reaction

```
# Create reaction object
reaction = Reaction(
    name="NH3+OH",
    reactantsPath=[reactant1, reactant2],
    productsPath=[product1, product2],
    transitionStatePath=ts,
    temperature=298.15
)

# Calculate reaction properties
reaction.run()
```

2.4 Step 4: Calculate Rate Constants

```
# TST rate constant
tst = TST(reaction)
k_tst = tst.calculateKTST(bi_or_uni='bi')

# Apply Eckart tunneling correction
eckart = eckartCorrection(reaction)
k_eckart = k_tst * eckart.kappaEckart

print(f"k_TST    = {k_tst:.3e} cm^3/molecule/s")
print(f"k_Eckart = {k_eckart:.3e} cm^3/molecule/s")
print(f"Tunneling correction factor = {eckart.kappaEckart:.2f}")
```

3 Temperature Range Calculation

```
import numpy as np

# Define temperature range
temperatures = np.arange(200, 1001, 100)

# Calculate for each temperature
for T in temperatures:
    rxn = Reaction(..., temperature=T)
    rxn.run()

    tst = TST(rxn)
    k = tst.calculateKTST()

    print(f"T = {T} K: k = {k:.3e}")
```

4 Available Tunneling Corrections

```
from RateKing.tunneling_corrections import (
    eckartCorrection,    # Most accurate
    wignerCorrection,    # Simplest, low T
    bell35Correction,    # Bell (1935)
    bell58Correction,    # Bell (1958)
    bell2TCorrection     # Bell 2T
)

# Apply any correction
correction = eckartCorrection(reaction)
kappa = correction.kappaEckart
k_corrected = k_tst * kappa
```

5 Common Tasks

5.1 Check Available Links in File

```
from RateKing import Molecule

total_links = Molecule.getAvailableLinks("file.out")
print(f"File contains {total_links} links")
```

5.2 Access Reaction Properties

```
reaction.run()

print(f"Ea = {reaction.activationEnergy:.2f} kcal/mol")
print(f"Delta E = {reaction.deltaE:.2f} kcal/mol")
print(f"Delta H = {reaction.deltaH:.2f} kcal/mol")
print(f"Delta G = {reaction.deltaG:.2f} kcal/mol")
print(f"Imaginary freq = {reaction.transitionStateImaginaryFrequency:.1f
    } cm^-1")
```

5.3 Calculate Dynamic TST

```
from RateKing import dTST

dtst = dTST(reaction)
k_dtst = dtst.calculateKDTST(bi_or_uni='bi')
```

6 File Requirements

Your Gaussian output files must include:

- Optimized geometry (Opt calculation)
- Frequency analysis (Freq keyword)
- Thermochemistry at desired temperature

- For TS: exactly one imaginary frequency

Example Gaussian input:

```
%NProcShared=8
%Mem=16GB
#P M062X/6-311++G(d,p) Opt Freq Temperature=298.15

Title

0 1
[geometry]
```

7 Troubleshooting

7.1 Import Error

```
pip install --upgrade RateKing
```

7.2 File Not Found

Use absolute paths or verify file locations:

```
import os
print(os.path.abspath("your_file.out"))
```

7.3 No Imaginary Frequency

Verify your TS file:

- Is it actually a transition state optimization?
- Did the frequency calculation complete?
- Is the link number correct?

8 More Information

- Full User Guide: `docs/UserGuide/RateKing_UserGuide.pdf`
- GitHub: https://github.com/phfmatias/RateKing
- Email: phfmatias@discente.ufg.br

Copyright © 2026 LEEDMOL. All rights reserved.